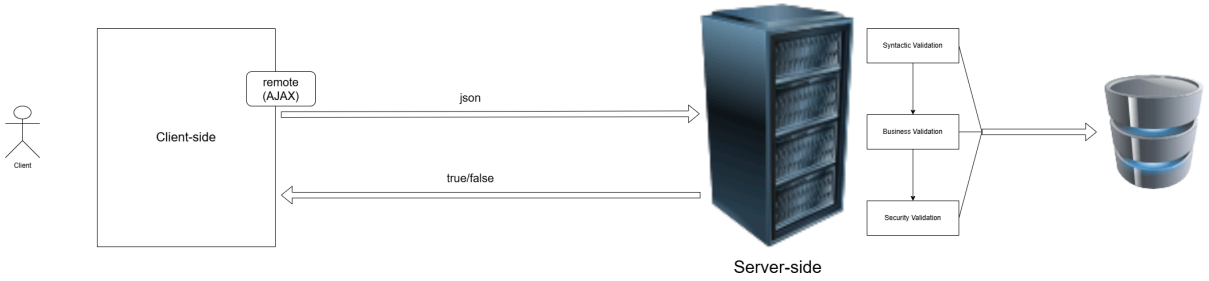
Research Validation



Server-side

Tại sao đã check JavaScript ở trình duyệt rồi, nhưng ta vẫn phải check ở Server nữa ?

1. Người dùng có thể tắt JavaScript
2. Có thể dùng các công cụ như Postman, cURL để gửi Request trực tiếp mà không thông qua giao diện
3. Hacker có thể thay đổi dữ liệu trên đường truyền

> 💡 Server-side là chốt chặn cuối cùng để bảo vệ Database và logic của hệ thống

Cách tạo một Request

Cách tạo một request để validate thay vì validate trực tiếp trong controller:

- Bước 1: Dùng artisan khởi tạo

```
php artisan make:request NameFormRequest
```

Đặt tên file request: [Action][Object][Request]

- Bước 2: Thêm validate cho các thuộc tính của request ở hàm rule()

```
public function rules(): array
    {
```

```
        return [
            'email' => 'required|email',
            'password' => 'required|string',
        ];
    }
```

- Bước 3: (Bonus) Nếu muốn thêm message cho các validate để trả về error ở hàm message

```
public function messages(): array
    {
        return [
            'email.required' => 'We need your email address
to log you in.',
            'email.email' => 'Please enter a valid email fo
rmat.',
            'password.required' => 'Password cannot be empt
y.',
        ];
    }
```

Các loại validation trong Backend

Syntactic Validation

Kiểm tra cú pháp / kiểu dữ liệu / format

```
'email' => 'required|email'
'age' => 'integer|min:18'
```

Business Validation

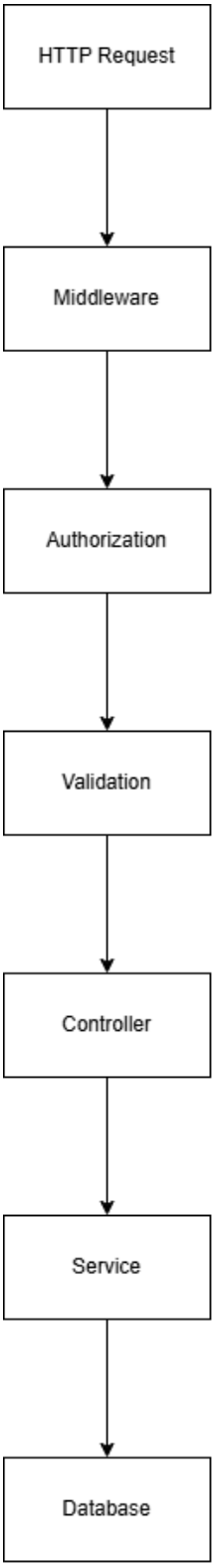
Kiểm tra luật nghiệp vụ

```php
if ($product->stock < $quantity)
```

Security Validation

```php
$this->authorize('update', $post);
```

Validation Flow chuẩn trong Lavarel



Thực hiện validate trong controller khi đã tạo định nghĩa validate ở request

```php
public function store(StorePostRequest $request)
{
    $validated = $request->validated();
}
```

Một số kỹ thuật nâng trong Validated

Bail

Dừng kiểm tra ngay sau khi gặp lỗi đầu tiên của một trường để tiết kiệm hiệu năng.

```php
'email' => 'bail|required|email|unique:users'
```

Nested data

Thường xuất hiện khi gửi một JSON phức tạp từ Frontend lên, ví dụ như một đơn hàng có nhiều sản phẩm, hoặc một danh sách các thuộc tính. Laravel xử lý việc này bằng kỹ thuật **Dot Notation**

```json
{
    "cart": [
        {"id": 1, "quantity": 2},
        {"id": 5, "quantity": 1}
    ]
}
```

Rule trong Lavarel

```php
public function rules(): array
{
    return [
        'cart' => 'required|array|min:1', // Giỏ hàng phải
là mảng và không được trống
        'cart.*.id' => 'required|integer|exists:products,i
d', // Mọi ID phải là số và tồn tại trong bảng products
```

```
        'cart.*.quantity' => 'required|integer|min:1', // M
ọi quantity phải >= 1
    ];
}
```

Custom Rules

Bước 1: Nếu các luật có sẵn không đủ, bạn có thể tự tạo

```
php artisan make:rule Uppercase
```

Bước 2: Viết logic kiểm tra

```
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class Uppercase implements ValidationRule
{
    /**
     * Run the validation rule.
     *
     * @param  \Closure(string): \Illuminate\Translation\Po
tentiallyTranslatedString  $fail
     */
    public function validate(string $attribute, mixed $valu
e, Closure $fail): void
    {
        // Logic: Nếu giá trị không bằng chính nó khi viết
hoa hoàn toàn
        if (strtoupper($value) !== $value) {
            // Gọi $fail để báo lỗi
            $fail("Trường :attribute phải được viết bằng ch
ữ IN HOA.");
        }
```

```
    }
}
```

Bước 3: Sử dụng trong Request

```
use App\Rules\Uppercase; // Đừng quên import

public function rules(): array
{
    return [
        'coupon_code' => ['required', 'string', new Upperca
se],
    ];
}
```

Client-side

Việc thêm **jQuery Validation** ở phía Client sẽ giúp trải nghiệm người dùng (UX) mượt mà hơn. Người dùng sẽ thấy lỗi ngay lập tức khi đang gõ mà không cần đợi trang load lại

Định nghĩa jQuery

Là một thư viện JavaScript với khẩu hiệu: **"Write less, do more"**

Nó giúp ta:

- Tìm các phần tử HTML cực nhanh
- Thay đổi nội dung, màu sắc, ẩn/hiện phần tử
- Xử lý sự kiện (Click, Hover, Submit)
- Gọi AJAX (**Asynchronous JavaScript And XML)** gửi ngầm request mà không cần reload lại trang (gửi dữ liệu lên Laravel mà không load lại trang).

Cách nhúng jQuery vào dự án qua CDN online

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.19.5/jquery.validate.min.js"></script>
```

💡 Thường sẽ nhúng vào file layout.blade.php trước thẻ </body>

@stack

Dùng để gom tất cả code JS của các file con lại và đặt chúng vào đúng vị trí **sau khi** các thư viện đã load xong.

```
<script src="jquery.min.js"></script>

@stack('scripts') {{-- Đây là nơi các đoạn code từ file con sẽ nhảy vào nằm --}}
</body>
```

@push

Dùng để push với đúng cái tên script đó

```
@section('content')
   @endsection

@push('scripts')
<script>
    console.log("Tôi là code của trang Login");
    $("#loginForm").validate();
</script>
@endpush
```

💡 Bằng cách dùng `@stack('scripts')` ở cuối layout (dưới link jQuery), chúng ta đảm bảo đoạn code `$("#loginForm").validate(...)` luôn luôn chạy sau khi jQuery đã sẵn sàng. Nếu không dùng stack, code của bạn sẽ chạy trước và trình duyệt sẽ báo lỗi: *"I don't know what `$` is!"*

Syntax

Mọi dòng code jQuery đều bắt đầu bằng dấu `$`. Cấu trúc là: $(selector).action()

- `$`: Ký hiệu của jQuery.
- `selector`: Tìm cái gì? (Giống hệt cách viết CSS).
- `action()`: Làm gì với thẻ đó

Ready event

```
$(document).ready(function() {
    // Toàn bộ code jQuery của bạn phải nằm ở đây
    console.log("Hệ thống đã sẵn sàng!");
});
```

Validate

```
<script>
    $(document).ready(function(){
        $('#loginForm').validate({
            rules: {
                email: {
                    required: true,
                    email: true
                },
                password: {
                    required: true
                }
            },
            messages: {
```

```
                email: {
                    required: "Vui lòng nhập email",
                    email: "Email không đúng định dạng"
                },
                password: {
                    required: "Vui lòng nhập mật khẩu"
                }
            },

            // Giúp hiển thị lỗi đẹp với Bootstrap 5
            errorElement: 'div',
            errorPlacement: function (error, element) {
                error.addClass('invalid-feedback');
                element.closest('.mb-3').append(error);
            },
            highlight: function (element) {
                $(element).addClass('is-invalid');
            },
            unhighlight: function (element) {
                $(element).removeClass('is-invalid');
            }
        });
    });
</script>
```

Remote

Để tăng trải nghiệm người dùng nếu email đã được đăng ký thì check mà không cần ấn submit và load lại trang

```
<script>
    $(document).ready(function() {
        // Thêm các hàm kiểm tra Regex cho jQuery
        $.validator.addMethod("strongPassword", function(va
lue) {
            return /[A-Z]/.test(value) // Chữ hoa
                && /[0-9]/.test(value) // Sô´
```

```
                && /[^A-Za-z0-9]/.test(value); // Ký tự đặc
biệt
        }, "Mật khâủ phải bao gôm̀ chữ hoa, sô´và ký tự đặc
biệt.");

        $("#registerForm").validate({
            rules: {
                name: {
                    required: true,
                    maxlength: 255
                },
                email: {
                    required: true,
                    email: true,
                    remote: {
                        url: "{{ route('check.email') }}",
                        type: "post",
                        data: {
                            _token: "{{ csrf_token() }}",
                            email: function(){
                                return $("input[name='emai
l']").val();
                            }
                        }
                    }
                },
                password: {
                    required: true,
                    minlength: 8,
                    strongPassword: true // Dùng hàm vừa tạ
o ở trên
                },
                password_confirmation: {
                    required: true,
                    equalTo: "input[name='password']"
                }
            },
```

```
            messages: {
                name: {
                    required: "Vui lòng nhập tên",
                    maxlength: "Độ dài tối đa 255 ký tự"
                },
                email: {
                    required: "Vui lòng nhập email",
                    email: "Email không đúng định dạng",
                    remote: "Email đã tồn tại"
                },
                password: {
                    required: "Vui lòng nhập mật khẩu",
                    minlength: "Mật khẩu phải ít nhất 8 ký
tự",
                },
                password_confirmation: {
                    required: "Vui lòng xác nhận lại mật kh
ẩu",
                    equalTo: "Mật khẩu không khớp"
                }
            },

            errorElement: 'div',
            errorPlacement: function (error, element) {
                error.addClass('invalid-feedback');
                element.closest('.mb-3').append(error);
            },
            highlight: function (element) {
                $(element).addClass('is-invalid');
                $(element).siblings('.text-danger').hide();
            },
            unhighlight: function (element) {
                $(element).removeClass('is-invalid');
            }
        });
    });
</script>
```

Custom số rule: char 1-byte/2-byte, regex expression,...

Regex Expression

Lavarel hỗ trợ Rule Regex

```
$request->validate([
    'username' => ['required', 'regex:/^[a-zA-Z0-9_]+$/']
]);
```

1-byte character (Half-Width)

1-byte character (Half-width)

- ASCII bình thường
- Ví dụ: A B C D 1 2 3 ! @ #

2-byte character (Full-Width)

- Ký tự full-width tiếng nhật / Unicode wide chars
- Ví dụ: A B C D 1 2 3 あ、い、う、え、お

Bảng so sánh

Tiêu chí	Half-width (1-byte)	Full-width (2-byte)
Cách hiểu đơn giản	Ký tự "hẹp", bình thường	Ký tự "rộng", chiếm nhiều chỗ hơn
Ví dụ chữ cái	`A B C`	`A B C`

Tiêu chí	Half-width (1-byte)	Full-width (2-byte)
Ví dụ số	`1 2 3`	`１ ２ ３`
Ví dụ ký tự đặc biệt	`@ - /`	`＠ － ／`
Cách gõ thường gặp	Bàn phím English / ASCII	IME tiếng Nhật / chuyển full-width
Dùng cho	Code, số, ID, phone, postal code	Tên tiếng Nhật, văn bản hiển thị/in ấn
Máy tính coi là	Ký tự chuẩn ASCII	Ký tự Unicode khác hoàn toàn
So sánh chuỗi	`"123"`	`"１２３"`
Có bằng nhau không?	`"123" === "123"` → true	`"123" === "１２３"` → false
Business rule thường gặp	"Phone phải half-width number"	"Tên khách hàng full-width only"

Bảng khi nào cần dùng Half-width

Loại dữ liệu	Thường dùng	Lý do
Email	Half-width	Theo chuẩn email / ASCII
Password	Half-width	Tránh lỗi encoding / auth
Username / Login ID	Half-width	Dễ compare / login
Phone Number	Half-width	Dữ liệu kỹ thuật / API
Postal Code	Half-width	Chuẩn hệ thống / partner
Customer ID / Account Code	Half-width	Machine-friendly
Product Code / SKU	Half-width	Search / compare chính xác
URL	Half-width	Theo chuẩn URL
IP Address	Half-width	Technical data
Số lượng / Giá tiền	Half-width	Tính toán / parse số

Bảng khi nào cần dùng Full-Width

Loại dữ liệu	Thường dùng	Lý do
Tên khách hàng (Japanese)	Full-width	Hiển thị đẹp / đúng chuẩn
Tên công ty tiếng Nhật	Full-width	In chứng từ / đồng bộ typography
Địa chỉ tiếng Nhật	Full-width	Theo chuẩn nhập địa chỉ
Furigana / Kana Name	Full-width Katakana	Business rule chuẩn Nhật

Loại dữ liệu	Thường dùng	Lý do
Ghi chú / Comment tiếng Nhật	Full-width	Natural Japanese text
Nội dung email tiếng Nhật	Full-width	Natural Japanese text

Tạo rule

FullWidth Rule

```
php artisan make:rule FullWidth
```

```php
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class FullWidth implements ValidationRule
{
    public function validate(string $attribute, mixed $valu
e, Closure $fail): void
    {
        if (!preg_match('/^[^\x01-\x7E\xA1-\xDF]+$/u', $val
ue)) {
            $fail("The {$attribute} must contain full-width
characters only.");
        }
    }
}
```

HalfWidth Rule

```
php artisan make:rule HalfWidth
```

```php
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class HalfWidth implements ValidationRule
{
    public function validate(string $attribute, mixed $valu
e, Closure $fail): void
    {
        if (!preg_match('/^[\x20-\x7E]+$/', $value)) {
            $fail("The {$attribute} must contain half-width
characters only.");
        }
    }
}
```

Auto Normalize Input

```php
protected function prepareForValidation(): void
{
    $this->merge([
        'email' => mb_convert_kana($this->email, 'as'),
        'password' => mb_convert_kana($this->password, 'a
s'),
        'username' => mb_convert_kana($this->username, 'ASK
V'),
    ]);
}
```

Các mode trong mb_convert_kana

Mode	Tác dụng
`n`	Full-width numbers → half-width

Mode	Tác dụng
`a`	Full-width alnum → half-width
`s`	Full-width spaces → half-width
`A`	Half-width alnum → full-width
`S`	Half-width spaces → full-width
`K` / `V`	Kana conversions

Chuyển <script></script> sang thành 1 file riêng

Bước 1: Tạo resources/js/register.js

```
$(document).ready(function() {
        // Thêm các hàm kiểm tra Regex cho jQuery
        $.validator.addMethod("strongPassword", function(va
lue) {
            return /[A-Z]/.test(value)
                && /[0-9]/.test(value)
                && /[^A-Za-z0-9]/.test(value);
        }, "Mật khẩu phải bao gồm chữ hoa, số và ký tự đặc
biệt.");

        $("#registerForm").validate({
            rules: {
                name: {
                    required: true,
                    maxlength: 255
                },
                email: {
                    required: true,
                    email: true,
                    remote: {
                        url: window.laravelData.checkEmailU
rl,
                        type: "post",
                        data: {
```

```
                                _token: "{{ csrf_token() }}",
                                email: function(){
                                    return $("input[name='emai
l']").val();
                                }
                            }
                        }
                    },
                    password: {
                        required: true,
                        minlength: 8,
                        strongPassword: true
                    },
                    password_confirmation: {
                        required: true,
                        equalTo: "input[name='password']"
                    }
                },

                messages: {
                    name: {
                        required: "Vui lòng nhập tên",
                        maxlength: "Độ dài tối đa 255 ký tự"
                    },
                    email: {
                        required: "Vui lòng nhập email",
                        email: "Email không đúng định dạng",
                        remote: "Email đã tồn tại"
                    },
                    password: {
                        required: "Vui lòng nhập mật khẩu",
                        minlength: "Mật khẩu phải ít nhất 8 ký
tự",
                    },
                    password_confirmation: {
                        required: "Vui lòng xác nhận lại mật kh
ẩu",
                        equalTo: "Mật khẩu không khớp"
```

```
                }
            },

            errorElement: 'div',
            errorPlacement: function (error, element) {
                error.addClass('invalid-feedback');
                element.closest('.mb-3').append(error);
            },
            highlight: function (element) {
                $(element).addClass('is-invalid');
                $(element).siblings('.text-danger').hide();
            },
            unhighlight: function (element) {
                $(element).removeClass('is-invalid');
            }
        });
    });
```

Bước 2: Đăng ký file js với file vite.config.js

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';

export default defineConfig({
    plugins: [
        laravel({
            input: ['resources/css/app.css',
                    'resources/js/app.js',
                    'resources/js/auth/register.js'],
            refresh: true,
        }),
    ],
});
```

Bước 3: Trong resources/views/auth/register.blade.php

```
@extends('layouts.app')

@section('title', 'Đăng ký')

@section('content')
<div class="row justify-content-center">
    <div class="col-md-6">
        <div class="card shadow">
            <div class="card-header bg-primary text-white t
ext-center"><h4>Đăng ký tài khoản</h4></div>
            <div class="card-body">
                <form action="{{ route('register.post') }}"
method="POST" id="registerForm">
                    @csrf <div class="mb-3">
                        <label class="form-label">Tên của b
ạn</label>
                        <input type="text" name="name" clas
s="form-control" value="{{ old('name') }}">
                        @error('name') <small class="text-d
anger">{{ $message }}</small> @enderror
                    </div>

                    <div class="mb-3">
                        <label class="form-label">Email</la
bel>
                        <input type="email" name="email" cl
ass="form-control" value="{{ old('email') }}">
                        @error('email') <small class="text-
danger">{{ $message }}</small> @enderror
                    </div>

                    <div class="mb-3">
                        <label class="form-label">Mật khẩu
</label>
                        <input type="password" name="passwo
rd" class="form-control">
                        @error('password') <small class="te
xt-danger">{{ $message }}</small> @enderror
```

```
                    </div>

                    <div class="mb-3">
                        <label class="form-label">Xác nhận
mật khẩu</label>
                        <input type="password" name="passwo
rd_confirmation" class="form-control">
                    </div>

                    <button type="submit" class="btn btn-pr
imary w-100">Đăng ký ngay</button>
                </form>
                <div class="text-center mt-3">
                    <a href="{{ route('login') }}">Đã có tà
i khoản? Đăng nhập</a>
                </div>
            </div>
        </div>
    </div>
</div>
@endsection

@push('scripts')
<script>
    window.laravelData = {
        checkEmailUrl: "{{ route('check.email') }}"
    };
</script>
@vite(['resources/js/auth/register.js'])
@endpush
```

Bước 4: Chạy lệnh npm

Vì file đang nằm trong `resources/js`, trình duyệt sẽ không đọc được trực tiếp. Bạn cần chạy lệnh sau ở Terminal của Laragon:

```
npm install
```

```
npm run dev
```

- `npm run dev`: Dùng khi bạn đang code (nó sẽ tự động cập nhật khi bạn sửa file).
- `npm run build`: Dùng khi bạn đã xong và muốn nén file lại để chạy thật.